

File Encoding Guide

A dedicated monograph on the forward half of the DataShield codec — transforming a file into a packet FEC stream. The document complements `AlgorithmGuide.en.md` (sections 2–4 give an overview): here the same topic is treated formally — the problem statement, definitions and invariants, step-by-step algorithms with justifications, output-volume estimates, worked examples, and the exact boundaries of the guarantees. An engineering retelling with matching section numbering is `EncoderGuide.Engineer.en.md`; a from-scratch tutorial is `EncoderGuide.Tutorial.en.md`. The reverse transformation is the `DecoderGuide.*` family; the arbitration of the result is the `AssemblyGuide.*` family.

1. Problem statement

Given the contents $F \in \{0,1\}^*$, $|F| = S$, and a name `name`. Construct an ordered sequence of packets $P = (P_1, \dots, P_L)$, $|P_i| = 75$, satisfying:

- Decodability.** From any sufficiently "surviving" subset of P the decoder recovers F bit for bit (the proof of correctness is a SHA-256 match; see `AssemblyGuide.Academic.en.md`).
- Self-verifiability.** Every packet is verifiable either autonomously (the header) or given the header of its own file (a sector).
- Refusal instead of defect.** Any constraint violation ends in an exception; partial output is impossible.

Redundancy parameters: `eccPercent p` $\in [0, 100]$, `headerPercent h` $\in [0, 100]$; defaults 10 and 3.

2. The loss model and redundancy

The channel damage model: (a) erasure of stream intervals; (b) garbage insertion and boundary desynchronization; (c) deliberate packet substitution. The answers: (a) — a Reed–Solomon erasure code and header repetition; (b) — the decoder's sliding window with step 1 (see `DecoderGuide.Academic.*.md`, sections 5–6); (c) — binding sectors with the `H5` seed and the final `H3` arbitration.

Definitions of the dimensions:

```
N = max(1, [S / 64])           (data volumes)
M = 0 if p = 0; else max(1, [N·p / 100]) (ECC volumes)
T = N + M                      (data sectors)
H = max(3, [T·h / 100])        (header copies)
L = T + H                      (packets in the stream)
```

All roundings are upward; `max(1, ·)` guarantees the non-triviality of the encoding and identification operations.

3. Format: constants and structures

A packet is 75 bytes. Header: $H1 \parallel H2 \parallel H3 \parallel H4 \parallel H5$, where $H1$ is the name (14 bytes, ASCII), $H2$ is the size S (3 bytes LE, $S \leq 2^{2^4}-1$), $H3$ is the SHA-256 of the contents (32), $H4$ is M (2 bytes LE), $H5$ is 24 bytes. Sector: $D1 \parallel D2 \parallel D3$, where $D1$ is the number (2 bytes LE), $D2$ is the payload (64), $D3$ is 9 bytes. The ceiling $T \leq 2^{1^6}-1$ stems from the $GF(2^{1^6})$ field size of the Reed–Solomon code.

Invariant I1 (dimension consistency). The header uniquely determines N , M , T from S and $H4$; the encoder and the decoder compute them by identical formulas, so inconsistent dimensions are impossible.

4. Data preparation

Steps: validate $S \leq 2^{2^4}-1$; pack the name $H1 = \text{Pack}(\text{name})$; $H3 = \text{SHA-256}(F)$; compute N, M, T with the check $T \leq 2^{1^6}-1$; slice F into payloads $d_0, \dots, d_{\{N-1\}}$ (the last one zero-padded: $d_{\{N-1\}} = F[(N-1) \cdot 64 .. S] \parallel 0^{N \cdot 64 - S}$).

Proposition 4.1 (injectivity of slicing). The map $F \mapsto (d_0, \dots, d_{\{N-1\}}, S)$ is injective: S is known from $H2$, so $F = [d_0 \parallel \dots \parallel d_{\{N-1\}}]_{0..S}$ is recovered by discarding the tail padding. ■

Name packing: the name is split at the first occurrence of a dot; the extension suffix is preserved whole; the base is truncated to budget with a $\sim$ marker; at a base budget < 2 — an exception (the name is unrepresentable). The full name is not transmitted: $H1$ serves as a human-readable label.

5. ECC encoding

A Reed–Solomon erasure code over $GF(2^{1^6})$ is used. A volume's payload is 32 field symbols (UInt16 LE). For each position $s \in [0, 32)$ independently: the column $c = (d_0[s], \dots, d_{\{N-1\}}[s]) \in GF(2^{1^6})^N$ is extended by the linear operator `RsRaid16.Process` to (c, e) with $e \in GF(2^{1^6})^M$; ECC volume j receives the symbol $e_j[s]$ at position s .

Proposition 5.1 (recovery condition). Let $A \subseteq [0, T)$ be the set of received volumes with $|[0, N) \cap A| = N - k$ (k data volumes lost) and $|[N, T) \cap A| \geq k$. Then all data volumes are recovered exactly. Proof: the Reed–Solomon scheme is a systematic MDS code with distance $M+1$; any N of the $N+M$ codeword components determine the word uniquely. Erasures (loss positions known from the map) exclude decoding ambiguity. ■

Corollary: lost ECC volumes do not affect recovery. The boundary condition $N + M \leq 2^{1^6}-1$ is the field size.

Invariant I2 (ECC determinism). Encoding is a pure function of $(d_0, \dots, d_{\{N-1\}}, N, M)$: identical input yields identical ECC volumes (given the fixed reference `RsRaid16`).

6. The header packet and H5

$H5 = \text{Trunc}_{24}(\text{SHA-256}(H1 \parallel H2 \parallel H3 \parallel H4))$. The header packet is $H1 \parallel H2 \parallel H3 \parallel H4 \parallel H5$.

Proposition 6.1 (autonomous verifiability). Header correctness is checked without external data:

$\text{Trunc}_{24}(\text{SHA-256}(\text{first 51 bytes})) = \text{last 24 bytes}$. The probability of falsely accepting a random packet is 2^{-192} . ■

Proposition 6.2 (seed binding). Knowledge of $H5$ is necessary to verify any sector (section 7); $H5$ cannot be derived from the sectors in reasonable time (SHA-256 strength). Hence a file's sectors form a hash-linked structure rooted at the header. ■

7. Data sectors and D3

For $i \in [0, T)$: $D1 = \text{LE16}(i)$, $D2 = \text{payload}_i$, $D3 = \text{Trunc}_9(\text{SHA-256}(H5 \parallel D1 \parallel D2))$ (input — 90 bytes).

Proposition 7.1 (substitution resistance). The probability of a random/foreign packet passing the check is 2^{-72} per packet; for a stream of L packets — $\approx L \cdot 2^{-72}$. Practically unattainable; targeted substitution requires a preimage of the truncated SHA-256 under a known seed. ■

Invariant I3 (coverage completeness). The payloads of sectors $0..T-1$ are exactly $\text{d0...d}_{\{N-1\}}$ and the ECC volumes; no payload is lost or duplicated when the stream is assembled.

8. Header placement

ArrangePackets: $P_1 = \text{header}$; $P_L = \text{header}$; intermediate copies are inserted after every $\lceil T/(H-1) \rceil$ -th sector ($\text{interval} = \max(1, T/(H-2+1))$, insertion at $(i+1) \bmod \text{interval} = 0$ while copies remain).

Proposition 8.1 (evenness). The distance between neighboring header copies in the stream is $\Theta(T/H)$ up to a constant; cutting out any contiguous interval shorter than $\text{interval} - 1$ sectors leaves at least one copy intact. ■ (The interval bounds follow from the insertion construction.)

9. Output formats and PacketIO

Binary: the concatenation of the packets, $75 \cdot L$ bytes. Base64: $\text{Convert.ToBase64String}(P_i)$ per line; since $75 \equiv 0 \pmod{3}$, the line length is exactly 100 and there are no padding characters.

Proposition 9.1 (expansion estimate). Binary: $75(T+H)/S$; as $S \rightarrow \infty$ with p, h fixed: $75/64 \cdot (1 + p/100)(1 + h/100) + o(1)$ (for the typical $10/3 \approx 1.326$). Base64: multiply by $4/3$ (≈ 1.768) plus one line break per packet. ■

The decorative frames $>[...][...][\text{SHA-256:}...]/<[...]$ are not part of the payload and are ignored by the decoder (the filter discards non-Base64 bytes).

10. Progress and cancellation

Global-scale phases: preparation `[0,10)`, ECC `[10,75)` (rescaled linearly over the 32 symbols via `ScaledProgress`), packetization `[75,100)`, terminal report `Done` (100). Whole-percent throttling via `ProgressThrottle`. The `CancellationToken` is checked at phase boundaries and inside the ECC symbol loop; cancellation throws `OperationCanceledException` — a partial result is never formed.

Invariant I4 (atomicity). `Encode` either returns the full list of `L` packets or throws; intermediate states are not observable from outside.

11. Statistics and memory hygiene

`EncodeWithStats` returns `EncodeStats(S, H3, N, M, L, H)`. After successful assembly the following are zeroed: all data payloads, all ECC payloads, the serialized header.

Invariant I5 (residue minimization). After `Encode` completes, only the following remain in the heap: the returned packets and buffers unreachable by the user. The number of copies of `F` in memory equals the number of sector packets (unavoidable) plus zero extras.

12. Stream input

`ReadStreamContent`: for a seekable stream — read the remainder `Length - Position` with a prior ceiling check; for a non-seekable one — a copy via `MemoryStream` with a post-check. The stream is not closed by the encoder. The full read is mandatory: both `H3` and RS require the whole of `F`.

13. Worked examples

13.1. A 1000-byte file, p=10, h=3

`N=16, M=2, T=18, H=3, L=20`; insertion interval `[18/2]=9`; stream `H D0...D8 H D9...D17 H`. Binary 1500 bytes (150%), Base64 2000 characters + 20 line breaks.

13.2. An empty file

`S=0 ⇒ N=1`: the single data volume is 64 zeros; `M≥1` at `p≥1`. The file exists, assembles, `H3 = SHA-256(∅)`.

13.3. A one-byte file, p=10

`N=1, M=1, T=2, H=3, L=5`. Redundancy 6400% — the price of the lower bound `M ≥ 1`.

13.4. The limiting file

$S = 16,777,215$: $N = 262,145 > 65,535$ even at $M=0$ — an exception on the $GF(2^{16})$ ceiling. Hence the practical ceiling at $p=0$: $S \leq 65,535 \cdot 64 = 4,194,240$ bytes (4 MiB minus 256 bytes); with ECC — less.

13.5. The name `my.documents.2024.xlsx`

Split at the first dot: base `my` , extension `.documents.2024.xlsx` (20 bytes) > 12 — the base budget is negative: an "unrepresentable" exception.

14. Parameters and limits

Parameter	Domain	Consequence of violation
<code>eccPercent</code>	$[0, 100]$ (effectively — while $T \leq 65,535$)	<code>ArgumentOutOfRangeException</code> at < 0
<code>headerPercent</code>	$[0, 100]$	<code>ArgumentOutOfRangeException</code> at < 0
<code>S</code>	$\leq 16,777,215$	<code>InvalidOperationException</code>
<code>T = N+M</code>	$\leq 65,535$	<code>InvalidOperationException</code>
Name	base ≥ 2 bytes after reserving the extension	<code>InvalidOperationException</code>

15. Edge cases

Summarized in sections 13.2–13.5 and the table in section 14. Additionally: `p=0` skips the ECC phase entirely (zero time and memory overhead); non-seekable input is allowed but requires an intermediate in-memory copy.

16. Summary of invariants and guarantees

Invariants: **I1** dimension consistency; **I2** ECC determinism; **I3** payload coverage completeness; **I4** Encode atomicity; **I5** memory residue minimization.

Guarantees:

- 1. A complete, correct stream or an exception (I4).
- 2. `H3` is the exact SHA-256 of the contents; the assembly arbitration is impeccable (Proposition 4.1).
- 3. The header is verifiable autonomously with false acceptance 2^{-192} (6.1); sectors — only via `H5` with 2^{-72} (7.1).
- 4. The loss of $\leq$ available-ECC data volumes is recoverable exactly (5.1).
- 5. A minimum of 3 header copies at even $\Theta(T/H)$ intervals (8.1).
- 6. Expansion estimate: $75/64 \cdot (1+p/100) (1+h/100)$ binary; $\times 4/3$ Base64 (9.1).

Numeric summary: packet 75 bytes; payload 64; H5 192 bits; D3 72 bits; $S \leq 16,777,215$; $T \leq 65,535$; expansion at (10,3) $\approx 132\%/176\%$.